UNIVERSIDAD NACIONAL DE INGENIERÍA
Facultad de Ingeniería Industrial y de Sistemas
SI806 · Desarrollo Adaptativo e Integrado del Software

TALLER 3 · DESARROLLO E INTEGRACIÓN DE UNA API REST

Documento Técnico

Luxxito Sports: app móvil y API REST de reservas de canchas deportivas

API REST propia en capas que decide disponibilidad, precios, stock de equipamiento, pagos verificados con Ticket QR y cancelaciones con política de 24 horas, consumida por una aplicación Flutter para Android.

Integrantes

Steven Cadillo
Cliente móvil, pagos, pruebas y demostración

Elvis Arboleda
Base de datos Supabase y despliegue

Juan Medina
Backend, Render e infraestructura

Repositorio en GitHub (rama main)
`https://github.com/Arbolito-ctrl/LuxxitoSports/tree/main`

API pública
`https://luxxito-sports-api.onrender.com/api`

Swagger UI (probar la API desde el navegador)
`https://luxxito-sports-api.onrender.com/api/docs`

Lima, noviembre de 2026

1. Fundamentación y análisis del problema

1. ¿Qué problema concreto resolverán? El alquiler de canchas deportivas en Lima se gestiona por WhatsApp y cuadernos: se venden dos veces el mismo turno, se cobra mal la tarifa nocturna (con iluminación), se presta más equipamiento (chalecos, balones, árbitros) del que existe, se pierden turnos mientras se revisa un Yape/Plin y no hay registro de devoluciones por cancelación.

2. ¿Quién lo necesita y por qué? El jugador u organizador necesita ver disponibilidad real desde su celular, apartar el turno 20 minutos mientras paga, agregar extras con stock verificado y llegar con un Ticket QR. El administrador del complejo necesita revisar comprobantes en un solo lugar, registrar el ingreso en puerta y aplicar reglas de cobro y reembolso sin discusiones.

3. ¿Qué hará la API que no deba hacer el cliente? El servidor es la única autoridad sobre disponibilidad, precios (diurno antes de las 18:00, nocturno desde las 18:00), seña del 50%, stock de extras por intervalo, emisión y uso único del Ticket QR, vencimiento de bloqueos y reembolsos de 24 horas. También decide quién puede hacer qué: el catálogo y la revisión de pagos son del ADMIN y cada reserva solo la ve y modifica su titular. La app Flutter muestra datos y solicita operaciones; nunca calcula un precio, no confirma una reserva por su cuenta y no se conecta directamente a la base de datos.

4. ¿Qué datos y relaciones se almacenarán? Siete tablas en PostgreSQL (Supabase): usuarios, complejos (1:N canchas y 1:N extras), canchas (1:N reservas), reservas (N:M extras mediante reserva_extras, 1:N pagos) y pagos. Los vouchers se guardan en el bucket privado "comprobantes" y las fotos de las canchas en el bucket público "canchas".

5. ¿Qué reglas ejecutará el servidor? (a) Cotización día/noche y seña del 50%; (b) exclusión mutua de turnos con bloqueo de 20 minutos y vencimiento automático; (c) stock de extras compartido por intervalo; (d) pago en revisión, aprobación idempotente y Ticket QR de un solo uso; (e) cancelación con 100% de reembolso con 24 horas o más y 50% de retención con menos; (f) titularidad de reservas y roles JUGADOR/ADMIN; (g) nombre de cancha único por complejo.

6. ¿Cómo evitar datos inconsistentes? Transacciones SQL (BEGIN/COMMIT/ROLLBACK) al crear reservas con extras; claves foráneas y restricciones CHECK; una restricción de exclusión (reservas_sin_solapamiento, btree_gist) que impide en la propia base dos reservas activas solapadas aunque lleguen al mismo milisegundo; un índice único para el nombre de cancha por complejo; actualizaciones atómicas para marcar el QR como usado; y una cola en el servidor que procesa en orden las operaciones sobre la misma fecha o reserva.

7. ¿Qué entradas deben rechazarse? DNI distinto de 8 dígitos, correos mal formados, tarifa nocturna menor a la diurna, imágenes que no sean JPEG/PNG/WEBP y rechazos de pago sin motivo (400); peticiones sin token o con token vencido (401); un JUGADOR que intenta usar funciones de ADMIN o la reserva de otra persona (403); horas pasadas, fuera de atención o de más de 4 horas (422); turnos ocupados, extras sin stock, QR ya usado o cancelado y nombres de cancha repetidos (409); imágenes de más de 5 MB (413); más de 10 consultas de DNI por minuto desde la misma IP (429).

8. ¿Cómo comprobarán cada resultado? Con 37 pruebas automatizadas (Vitest + Supertest, npm test) que cubren éxitos, fallos, seguridad y concurrencia; con la colección Postman ejecutada por Newman (38 solicitudes y 63 aserciones aprobadas); y con una demostración completa en un Motorola Edge 50 usando el APK de producción contra la API de Render y la base de Supabase, con 12 capturas y el estado final de la base como evidencia.

2. Arquitectura en capas

- Cliente móvil (mobile/, Flutter + Dart): registro con sugerencia del nombre a partir del DNI; jugador con Explorar (filtros por distrito y deporte, foto de cada cancha), Disponibilidad y Reserva (turnos, extras y cotización del servidor), Mis Reservas (pago Yape/Plin/Transferencia con foto del voucher, Ticket QR, edición y cancelación) y Perfil. El ADMIN tiene su propio panel: Inicio, Pagos (bandeja con foto del comprobante), Ingreso (lector de QR con cámara o código escrito), Reservas y gestión de canchas. La sesión se guarda cifrada (flutter_secure_storage), la app solo se comunica con la API (services/api/, sin claves de Supabase) y se actualiza desde GitHub Releases.
- Rutas y middlewares (backend/src/routes, middlewares): definen los endpoints y aplican requireAuth (JWT), requireRole("ADMIN") y requireTitularDeReserva antes de llegar al controlador.
- Controladores (backend/src/controllers): leen la petición, llaman al servicio y traducen errores de dominio a códigos HTTP con un formato único { ok, code, message }.
- Servicios (backend/src/services): AuthService, CatalogService, DniService y el módulo reservas/ (CotizacionService, ReservaService, PagoService, ComprobanteService y TicketQrService) contienen todas las reglas de negocio y validaciones.
- Repositorios (backend/src/repositories): encapsulan SQL y transacciones contra PostgreSQL (pooler de Supabase) y el acceso a Storage. Un modo de almacenamiento local permite correr las pruebas sin base de datos.
- Infraestructura: API en Render (Node 22, despliegue automático desde main), base y buckets en Supabase, Decolecta para el nombre por DNI, y GitHub Actions para pruebas, despliegue y APK firmado con una clave fija.

3. Modelo de datos relacional (Supabase PostgreSQL)

Tabla 1. *Tablas del modelo relacional, campos y restricciones.*

Tabla	Campos principales	Relaciones y restricciones
usuarios	id, nombre, dni, telefono, email, password_hash (scrypt), rol (JUGADOR/ADMIN)	email y dni únicos
complejos	id, nombre, distrito, direccion, telefono, hora_apertura, hora_cierre, activo	1:N canchas, 1:N extras
canchas	id, complejo_id, nombre, deporte, superficie, techada, iluminacion, precio_hora_diurno, precio_hora_nocturno, estado, imagen_url	FK complejos; nocturno >= diurno; nombre único por complejo
extras	id, complejo_id, nombre, descripcion, precio_unitario, stock_total, activo	FK complejos; stock_total >= 0
reservas	id, cancha_id, cliente_nombre, cliente_dni, cliente_telefono, cliente_email, fecha, hora_inicio, hora_fin, subtotales, total, monto_pagado, monto_devolucion, estado, codigo_qr, qr_usado_at, expires_at	FK canchas; exclusión de solapamiento entre reservas activas
reserva_extras	id, reserva_id, extra_id, cantidad, precio_unitario, subtotal	FK con ON DELETE CASCADE; UNIQUE(reserva_id, extra_id)
pagos	id, reserva_id, metodo (YAPE/PLIN/TRANSFERENCIA), modalidad (SENA_50/TOTAL), monto, numero_operacion, comprobante_url, estado, observacion	FK reservas con ON DELETE CASCADE

Estados de una reserva: PENDIENTE_PAGO (bloqueo de 20 minutos) » EN_REVISION (comprobante enviado, ya no vence) » CONFIRMADA (pago aprobado, QR emitido). También puede pasar a CANCELADA o VENCIDA. Un pago va de PENDIENTE_REVISION a APROBADO, RECHAZADO o REEMBOLSADO.

Script de la base: backend/sql/schema.sql crea las tablas, restricciones, índices, políticas RLS y los datos iniciales.

4. Endpoints de la API REST propia

Base: `https://luxxito-sports-api.onrender.com/api` · JSON UTF-8 · 31 endpoints · contrato OpenAPI 3.0.3 versión 1.8.0 · Swagger UI público para probarlos: `https://luxxito-sports-api.onrender.com/api/docs`.

Tabla 2. *Endpoints propios, nivel de acceso y códigos HTTP.*

Método y ruta	Acceso	Función	Códigos
POST /auth/registro	Público	Crea una cuenta JUGADOR y devuelve el JWT	201, 400, 409
POST /auth/login	Público	Inicia sesión y devuelve el JWT (30 días)	200, 400, 401
GET /auth/me	Sesión	Perfil del usuario del token	200, 401
POST /auth/recuperar	Público	Solicita recuperar el acceso por correo	200, 400
GET /dni/:numero	Público	Sugiere el nombre del DNI en el registro (solo nombre)	200, 400, 404, 429, 503
GET /health · /version · /apk/download	Público	Estado, versión desplegada, última versión y descarga del APK	200, 302, 502, 503
GET /docs · /openapi.yaml	Público	Swagger UI y contrato OpenAPI	200
GET /complejos · /complejos/:id	Público	Complejos, con sus canchas y extras	200, 400, 404
GET /canchas · /canchas/:id	Público	Catálogo con filtros de complejo, distrito, deporte y estado	200, 400, 404
POST /canchas	ADMIN	CRUD Create: registra una cancha validando tarifas	201, 400, 401, 403, 409
PUT /canchas/:id	ADMIN	CRUD Update: tarifas, estado o datos de la cancha	200, 400, 401, 403, 404, 409
POST /canchas/:id/imagen	ADMIN	Sube la foto al bucket público canchas	200, 400, 401, 403, 404, 413
GET /canchas/:id/disponibilidad	Público	Regla 1: turnos de la fecha, tarifas y stock de extras	200, 400, 404
POST /reservas/cotizar	Público	Regla 2: precio de cancha + extras y seña del 50%	200, 400, 404, 409, 422
GET /reservas	Sesión	CRUD Read: reservas con extras y pagos (el jugador ve las suyas)	200, 401
GET /reservas/:id	Titular	Detalle de una reserva	200, 401, 403, 404
POST /reservas	Público	CRUD Create + Regla 3: bloqueo de 20 min sin solapamiento	201, 400, 404, 409, 422
PUT /reservas/:id	Titular	CRUD Update: teléfono o notas de una reserva activa	200, 400, 401, 403, 404, 409
DELETE /reservas/:id	Titular	CRUD Delete: elimina una reserva no confirmada	200, 401, 403, 404, 409
POST /pagos/comprobante	Público	Sube la foto del voucher al bucket privado	201, 400, 404, 409, 413, 502
POST /reservas/:id/pagos	Titular	Regla 4: registra el pago y pasa la reserva a revisión	201, 400, 401, 403, 409, 422
GET /pagos?estado=	ADMIN	Bandeja de comprobantes con enlace temporal a la foto	200, 400, 401, 403
PATCH /pagos/:id/aprobar	ADMIN	Regla 5: aprobación idempotente y emisión del QR	200, 401, 403, 404, 409
PATCH /pagos/:id/rechazar	ADMIN	Rechazo con motivo; la reserva vuelve a pago con 20 min	200, 400, 401, 403, 404, 409
POST /reservas/verificar-qr	ADMIN	Registra el ingreso en puerta; el QR sirve una sola vez	200, 400, 401, 403, 404, 409
POST /reservas/:id/cancelar	Titular	Regla 6: cancelación con política de 24 h y QR revocado	200, 401, 403, 404, 409

Formato de error: todas las respuestas de error usan { "ok": false, "code": "CONFLICT", "message": "..." }. "Titular" significa que el DNI del token coincide con el de la reserva o que el usuario es ADMIN.

5. Reglas de negocio en el servidor

5.1. Cotización día/noche: El servidor recorre cada hora de [hora_inicio, hora_fin): antes de las 18:00 cobra precio_hora_diurno y desde las 18:00 precio_hora_nocturno. Suma los extras y calcula monto_sena_50 = round(total × 0.50, 2). Ejemplo: 17:00-19:00 con 2 chalecos = S/ 90 + S/ 120 + S/ 30 = S/ 240.

5.2. Exclusión mutua y bloqueo de 20 minutos: Una reserva nueva se rechaza con 409 si otra PENDIENTE_PAGO, EN_REVISION o CONFIRMADA de la misma cancha y fecha se cruza (inicio < nuevo_fin y fin > nuevo_inicio). La reserva queda PENDIENTE_PAGO con expires_at = ahora + 20 min; si no llega el comprobante, pasa a VENCIDA y libera el turno. Con el comprobante enviado (EN_REVISION) ya no vence.

5.3. Stock de extras por intervalo: El equipamiento es del complejo y se comparte entre sus canchas. El servidor suma lo reservado en todas las reservas activas que se cruzan con el horario pedido; si lo solicitado supera el stock restante, responde 409.

5.4. Pago, aprobación idempotente y Ticket QR: El monto debe ser el 50% (SENA_50) o el 100% (TOTAL). Aprobar dos veces no duplica monto_pagado: la segunda llamada devuelve el mismo resultado. La primera aprobación confirma la reserva y genera LUXXITO-QR-<id>-<hash>. En puerta, el ADMIN lo escanea: la primera lectura registra el ingreso (qr_usado_at) y las siguientes responden 409.

5.5. Cancelación de 24 horas: Con la hora de Lima (UTC-5): con 24 horas o más de anticipación se reembolsa el 100% de lo pagado; con menos se retiene el 50%. En ambos casos se revoca el QR y se libera el turno.

6. Seguridad y concurrencia

Autenticación: JWT firmado con HMAC-SHA256 (JWT_SECRET) que incluye id, dni, rol y exp (30 días). Un token ausente, alterado o vencido responde 401.

Autorización: requireRole("ADMIN") protege el catálogo, la bandeja de pagos, aprobar, rechazar y verificar QR (un JUGADOR recibe 403). requireTitularDeReserva permite ver, editar, pagar, eliminar o cancelar una reserva solo a su titular o al ADMIN, y el listado de un JUGADOR devuelve solo sus reservas.

Contraseñas: Se guardan con scrypt, sal aleatoria de 16 bytes y hash de 64 bytes (scrypt$<sal>$<hash>), y se comparan en tiempo constante. Las cuentas antiguas en texto plano se migran a hash en su primer inicio de sesión.

Archivos: Los vouchers van a un bucket privado y el ADMIN los ve con un enlace firmado de 1 hora. Solo se aceptan JPEG, PNG o WEBP de hasta 5 MB.

Pedidos simultáneos: Si dos personas piden el mismo turno al mismo tiempo, la restricción de exclusión de PostgreSQL garantiza que solo una reserva se guarde; la otra recibe 409 "El turno elegido acaba de ser reservado por otra persona". Una cola en el servidor procesa en orden las operaciones de la misma fecha o reserva (aprobar, rechazar, cancelar, verificar QR), y el QR se marca como usado con una actualización atómica. Se usó una cola porque los bloqueos consultivos de PostgreSQL no funcionan con el pooler de transacciones de Supabase.

Secretos: Las credenciales (base de datos, JWT y el token de Decolecta) viven solo en Render, en GitHub Secrets y en un backend/.env local ignorado por Git. El APK solo recibe la URL de la API. En Supabase, la clave pública únicamente puede leer el catálogo: reservas, pagos, usuarios y las funciones rpc_ están cerrados. La consulta de DNI devuelve solo nombre y apellidos y tiene un límite de 10 por minuto por IP.

7. Pruebas automatizadas

Vitest + Supertest (backend/src/tests/): 37 de 37 pruebas aprobadas (29 en api.test.ts y 8 en dni.test.ts). Se ejecutan en cada push con GitHub Actions.

Tabla 3. *Casos de prueba automatizados agrupados por funcionalidad.*

Grupo	Casos y lo que comprueban
Catálogo y CRUD	1 filtros (200); 2 crear y editar cancha (201/200); 3 tarifa nocturna menor (400); 24 solo ADMIN y foto (401/403/400/404); 26 nombre repetido (409)
Cotización y reservas	4 reserva día+noche con extras (201); 5 turno solapado (409); 6 extras sin stock (409); 12 horas pasadas (422); 13 fuera de atención o más de 4 h (422); 14 seña 50%; 16 filtros e IDs (400/404); 20 vencimiento a los 20 min
Pagos y QR	7 pago y aprobación idempotente con QR; 9 bandeja; 10 rechazo y reintento; 11 rechazos inválidos (400/404/409); 19 QR de un solo uso (200/404/409); 21 y 22 voucher en el bucket privado
Cancelación	8 reembolso 100% con más de 24 h; 15 retención 50% con menos de 24 h
Seguridad	17 login, credenciales inválidas (401) y registro duplicado (409); 18 roles (401/403); 23 token vencido (401); 27 titularidad (401/403); 28 contraseñas con scrypt
Concurrencia	25 pedidos simultáneos no duplican turnos, extras, comprobantes ni aprobaciones
Documentación y DNI	29 Swagger UI y contrato OpenAPI; dni.test.ts: solo nombre y apellidos, caché, DNI inválido (400), no encontrado (404), sin token o proveedor caído (503) y límite por IP (429)

Postman / Newman (docs/LuxxitoSports.postman_collection.json): 38 solicitudes y 63 aserciones aprobadas: autenticación, catálogo, disponibilidad, cotización, reserva, foto del voucher, pago, aprobación, QR, cancelación y casos de error.

8. Despliegue

- API: Render (plan gratuito, Node 22). Se compila con npm run build, arranca con node dist/server.js y se verifica con GET /api/health (db_mode POSTGRES_POOL, db_ready true). Se despliega automáticamente desde main.
- Base de datos: Supabase PostgreSQL vía el pooler de transacciones, con buckets "comprobantes" (privado) y "canchas" (público).
- App: GitHub Actions compila y firma el APK y lo publica en Releases (latest-apk para producción, dev-apk para pruebas). La app avisa y se actualiza cuando hay una versión nueva.

9. Demostración del flujo completo

Equipo y entorno: Motorola Edge 50 con el APK de producción, contra la API pública de Render y la base de Supabase.

Recorrido: explorar y filtrar canchas » cotización del servidor » reserva creada con bloqueo de 20 min » el mismo turno ya aparece ocupado y otro intento recibe 409 » pago Yape con foto del voucher (EN_REVISION) » el ADMIN ve el comprobante en su bandeja y aprueba » Ticket QR » ingreso registrado en puerta » segundo escaneo rechazado (QR ya usado) » cancelación con la política de 24 horas.

Evidencia: docs/validaciones/DEMOSTRACION_TALLER3.md, 12 capturas en docs/validaciones/capturas/ y el estado final de la base en evidencia-supabase-demo.json. En las capturas se ocultaron datos personales de terceros.

Código fuente: https://github.com/Arbolito-ctrl/LuxxitoSports/tree/main (rama main).